nntrainer 백엔드 구조 지도

지금 코드가 어떻게 생겼는지, `ARCHITECTURE_REFACTOR.md`가 그리는 목표 모양, 그리고 둘 사이에 남은 일.

기준 tree: upstream main `4cc8dd5cd` + qc_npu 720 commits 목표 문서: `docs/backend_guide/ARCHITECTURE_REFACTOR.md`

1 현재 구조 현재 class diagram 2 목표 구조 목표 class diagram layer에 남는 것 backward도 op로 prefill · decode 한 경로 3 비교와 남은 일 backend별 불일치

지금	실행 사슬 **Engine → Context → ComputeOps → kernel**은 backend 5개 모두에 이미 있습니다. 문제는 "어느 backend로, 어떻게 돌릴지"를 정하는 곳이 **env 변수 275개, layer fork 11개, HTP 전용 모델 class**로 흩어져 있다는 점입니다.
목표	결정은 **compile 때 한 번** (ModelFeatures × DeviceCaps → ExecPlan). layer는 **설계도만** 맡고, 계산은 **backend 이름이 없는 op 목록**이 합니다.
남은 일	**rmsnorm · attention virtual 추가, ExecPlan이 실제로 결정하게 만들기**, 그 둘 위에서 **HTP를 공통 graph로 합치기**. 학습 쪽은 그다음 **backward도 op 목록으로**.

1 / 3

현재 구조

세로는 호출 순서, 가로는 backend입니다. 빨간 점선 칸은 **backend 차이가 op table 위(모델 · layer)로 새어 나온 곳**이고, 목표 구조에서는 사라져야 합니다.



backend 차이가 모델 · layer로 샌 곳 **빨간 글씨** 목표 규칙에 어긋나거나 빠진 것 --- 목표 경계 (위는 backend 중립)

그림 1. 아래 4줄(Context ~ kernel)은 이미 목표 모양입니다. 문제는 위 두 줄: OpenCL · CUDA는 layer를 fork하고, HTP는 모델 class 자체가 따로 있습니다.

→ **아래쪽은 잘 되어 있음.** backend마다 Context · ComputeOps · MemAllocator 한 벌씩. HTP와 CUDA는 `CpuComputeOps`를 상속해서 못 하는 op는 CPU로 떨어집니다.

→ **위쪽에서 backend가 갈라짐.** 같은 op인데 backend마다 layer class가 따로 있고(11개), 제일 큰 layer인 `mha_core.cpp`는 안에서 `#if`로 OpenCL · CUDA를 나눕니다.

→ **분기 결정은 그림 밖에 있음.** 어떤 kernel · 메모리 · decode 경로를 쓸지는 env 변수와 `#if`가 정합니다. 그림 6에서 자세히.

현재 class diagram

코드베이스의 class는 수백 개라서, backend 구조에 걸리는 class만 세 장으로 나눴습니다: 실행 사슬, graph · layer, 모델. 각 그림은 화면 한 장에 들어가도록 그렸습니다.

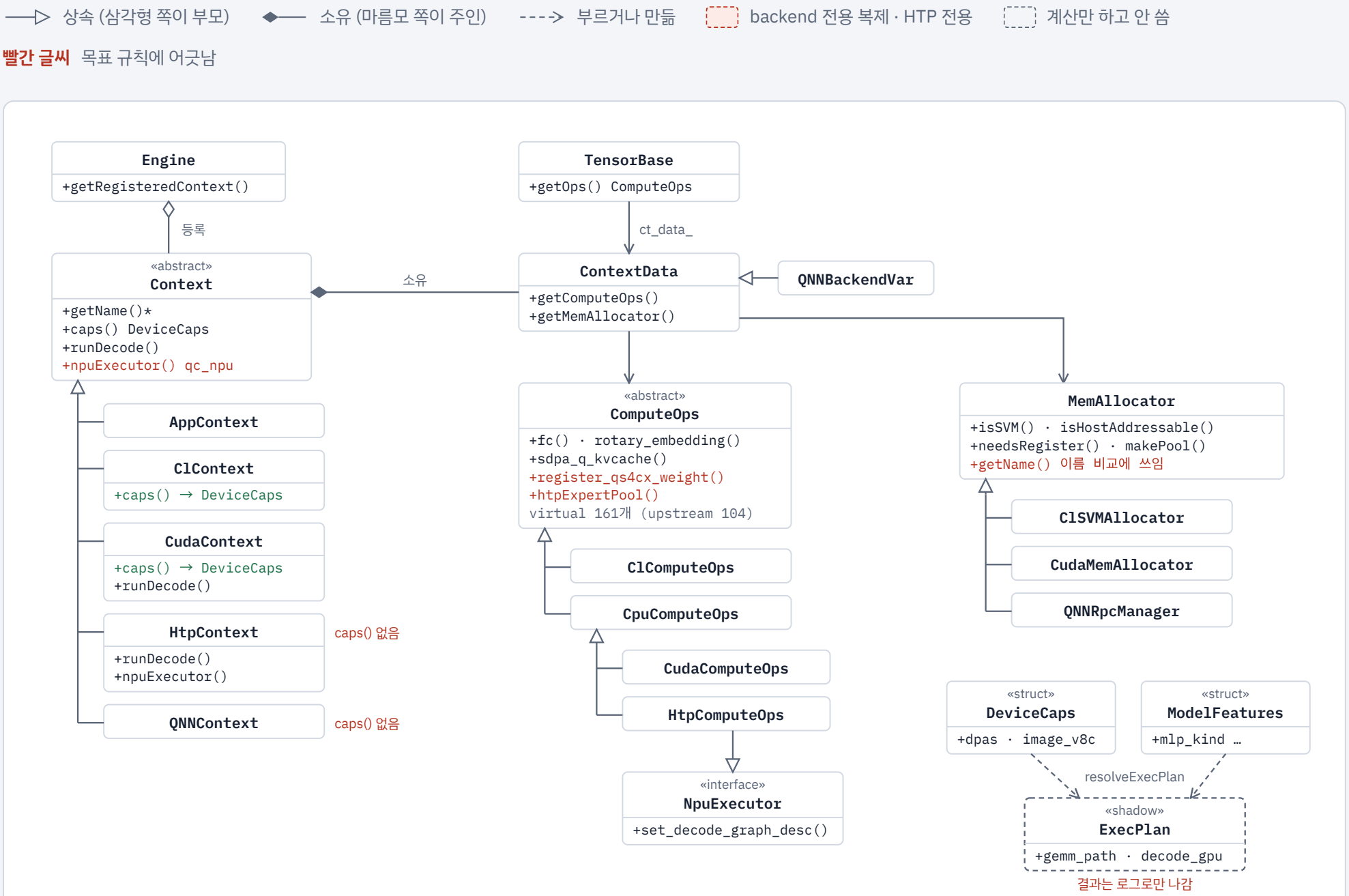


그림 2a. 실행 사슬. Context 5개가 각자 ContextData(op table + allocator)를 갖습니다. `ComputeOps`의 virtual은 upstream 104개에서 qc_npu 161개로 늘었고, `htpExpertPool()`처럼 vendor 이름이 붙은 것도 들어왔습니다. `ExecPlan`은 계산만 되고 쓰이지 않습니다. `npuExecutor()`는 qc_npu가 base Context에 추가했습니다.

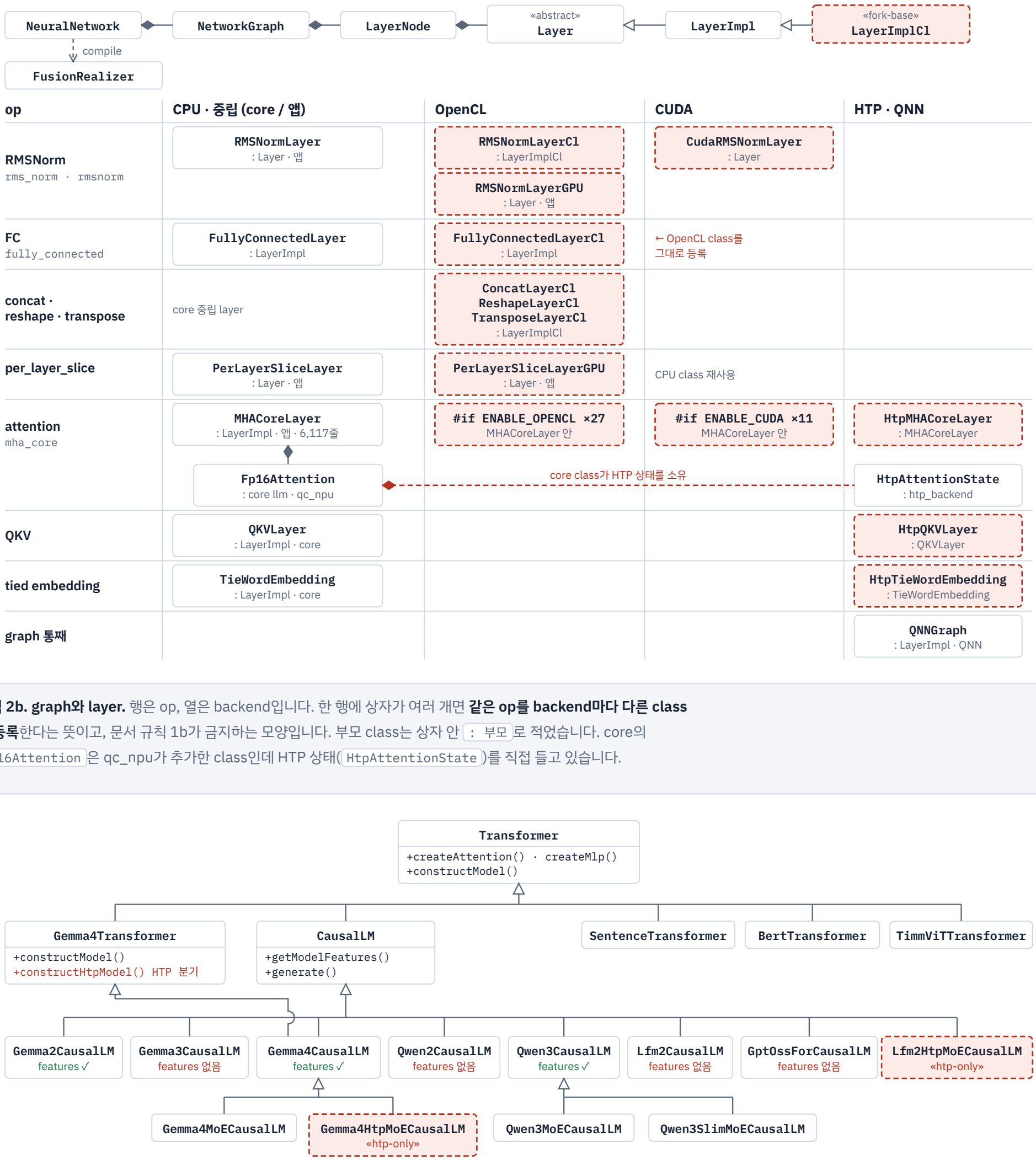


그림 2b. graph와 layer. 행은 op, 열은 backend입니다. 한 행에 상자가 여러 개면 **같은 op를 backend마다 다른 class로 등록**한다는 뜻이고, 문서 규칙 1b가 금지하는 모양입니다. 부모 class는 상자 안 `: 부모`로 적었습니다. core의 `Fp16Attention`은 qc_npu가 추가한 class인데 HTP 상태(`HtpAttentionState`)를 직접 들고 있습니다.

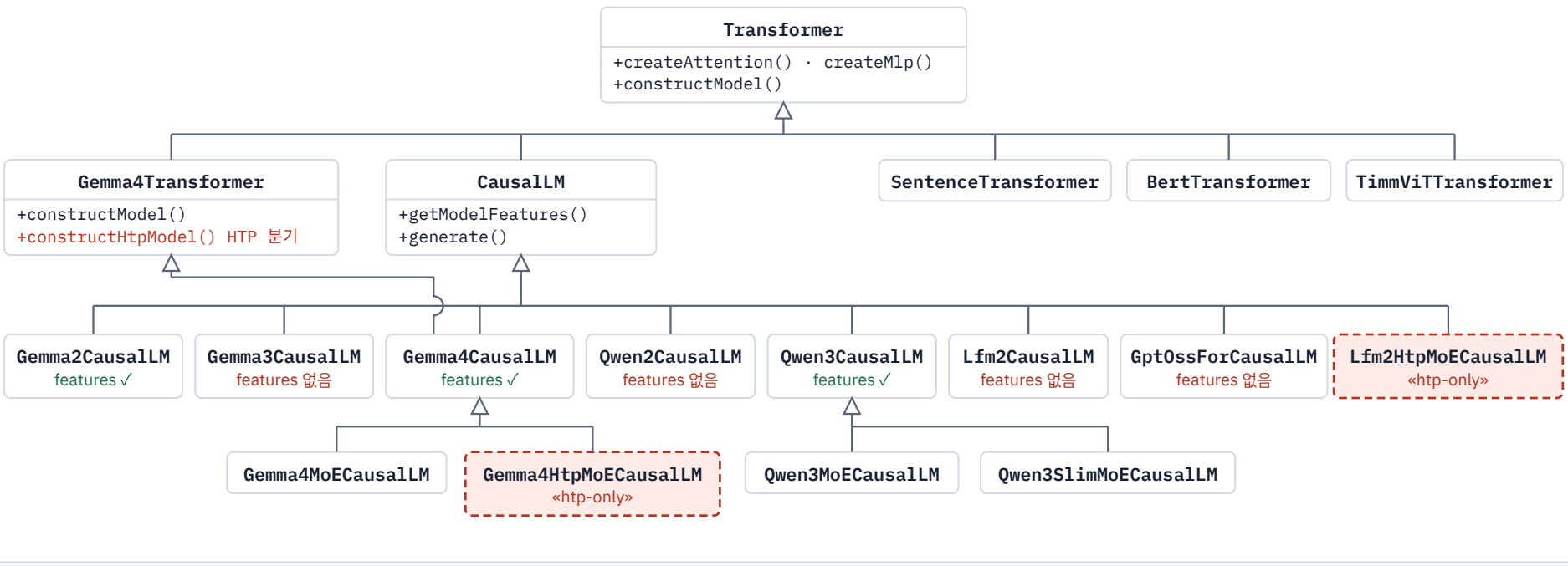


그림 2c. 모델. 각 `{Model}CausalLM`은 `CausalLM`과 `{Model}Transformer`를 같이 상속합니다(그림에는 Gemma4만 표시). 캐시 변형 모델 몇 개는 뺐습니다. `getModelFeatures()`를 채운 모델은 gemma2 · gemma4 · qwen3 계열뿐입니다. Gemma4는 `constructModel()` 안에서 `HTP_GRAPH` 플래그로 `constructHtpModel()`로 갈라지고, HTP 전용 subclass도 따로 있습니다.

목표 구조

그림 1과 같은 표를 목표 기준으로 다시 그렸습니다. **파란 칸이 지금과 달라지는 곳**이고, 위쪽 네 줄(모델 · 실행 계획 · layer 설계도 · op 목록)은 backend와 상관없이 하나만 남습니다.

	CPU engine=cpu	OpenCL engine=gpu	CUDA engine=cuda	HTP (NPU) engine=htp	QNN (NPU) engine=qnn
Model 모델 graph	**공통 모델 graph 하나 + ModelFeatures 선언, backend 이름 없음** Transformer → {Model}CausalLM. Gemma4 · LFM2의 HTP 전용 class는 여기로 합쳐짐				
Plan 실행 계획 (L4)	**compile 때 ExecPlan 한 번: resolve(DeviceCaps × ModelFeatures × ExecMode)** → step 모양(M)마다 kernel · 메모리 위치 · 실행 방식, 학습 가능 여부. 모든 backend가 같은 입구				
Layer 설계도	**layer = 설계도만: 속성 · shape · weight · 상태 선언, 그리고 lower()** rms_norm · mha_core · fc · qkv … op당 class 하나, 계산 코드는 없음 backend fork 11개와 layer 안 #if · engine 문자열 분기가 모두 사라짐				
Ops 실행 목록	**op 목록 하나 (prefill · decode 공용): forward는 lower()로, backward는 규칙표로 (2단계)** op 하나 = ComputeOps 호출 하나. fusion과 메모리 수명 계산도 이 목록 위에서				**OffloadNode** 목록을 통째로 claim
					↑ 이 선 위에는 backend 이름이 없음
Context backend 등록	`AppContext` caps(): 기본값	`ClContext` caps() 있음	`CudaContext` caps() 있음	`HtpContext` caps() 추가	`QNNContext` caps() 추가
ComputeOps 연산 table	`CpuComputeOps` + rmsnorm · attention	`ClComputeOps` + rmsnorm · attention arm	`CudaComputeOps` + attention · RoPE arm	`HtpComputeOps` 연산만, 자원 관리는 밖으로	**없음** 목록은 OffloadNode가
MemAllocator 메모리 위치	**기본 host**	`ClSVMAllocator` KV도 predicate로 판단	`CudaMemAllocator`	`rpcmem MemAllocator` needsRegister() 등 구현	`QNNRpcManager` predicate 구현
Run 실행 방식	**eager** M과 상관없이 op 단위	**eager** coherence는 caps로	**eager · capture** M=1 반복이면 replay	**op 단위 · 상주 graph** M=1 반복이면 상주 graph	**offload** M별 graph 자동 생성
Train 학습 (2단계)	**backward kernel** rmsnorm · attention 추가	**지원 op만** 없는 op면 Plan이 거절	**지원 op만** 없는 op면 Plan이 거절	**학습 없음** Plan이 학습 모드 거절	**학습 없음** Plan이 학습 모드 거절
Kernel 실제 계산	**NEON · AVX2**	**OpenCL kernel**	**CUDA kernel**	**HVX · HMX skel** FastRPC · dspqueue	**QNN SDK** dlopen

지금과 달라지는 칸 --- 목표 경계 (위는 backend 중립)

그림 3. 목표 구조, backend별. 그림 1의 빨간 칸(모델 · layer의 fork, layer 안 분기)이 모두 위쪽 공통 줄로 합쳐집니다. layer는 설계도만 남고, 계산은 compile 때 layer가 펼쳐 낸 op 목록이 맡습니다. Train 줄은 2단계(backward도 op 목록) 기준입니다. 실행 방식(Run) 줄은 prefill · decode를 나누지 않고, Plan이 step 모양(M)을 보고 고릅니다. backend 차이는 경계선 아래(Context · ComputeOps · 메모리 · decode mode · kernel)에만 남습니다. HTP 메모리 칸의 "rpcmem MemAllocator"는 문서 § 5 4단계를 따른 가칭입니다.

층으로 보면 규칙은 하나입니다. **능력(무엇을 할 수 있나)은 아래에서 위로, 결정(무엇을 쓸까)은 위에서 아래로** 흐르고, 어떤 층도 위층을 부르지 않습니다. 그림 4의 파란 칸 3개가 문서가 새로 더하는 층입니다.

`L5 MODEL`	중립 graph 하나 + ModelFeatures. layer는 설계도만, 계산 없음 지금: ModelFeatures는 3개 모델만 선언, HTP는 전용 class	일부
`L4 RESOLVER`	DeviceCaps × ModelFeatures × ExecMode → ExecPlan (순수 함수) 지금: 계산하고 로그만 찍음, 읽는 곳 없음	로그만
`L3a OP TABLE`	ComputeOps whole-op virtual, fused 포함 지금: rmsnorm · attention virtual 없음	일부
`L3b EXEC`	SEAM-1 op 목록 실행 (fwd · bwd · offload) · SEAM-2 step hook (runStep) 지금: layer가 직접 계산, op 목록 없음. hook은 runDecode 이름으로 있음	일부
`L2 DEVICECAPS`	이 하드웨어가 할 수 있는 것 지금: ClContext · CudaContext만 caps() 제공	일부
`L1 CONTEXT`	link 시점 자기 등록, QNN은 dlopen 지금: cpu · gpu · cuda · htp · qnn 등록됨	있음
`L0 HW PROBES`	clGetDeviceInfo, vendor device query 지금: OpenCL · CUDA가 caps를 여기서 채움	있음

결정 위 → 아래

능력 아래 → 위

문서가 새로 더하는 층 (L4, L2, SEAM-2)

그림 4. 오른쪽 알약은 qc_npu_rebase에서 그 층이 어디까지 있는지입니다. 완전히 없는 층은 없고, 대부분 "있지만 아직 결정을 안 함" 상태입니다. L3b의 "op 목록 실행"은 문서보다 한 걸음 더 나간 이 페이지의 안입니다. 문서는 얇은 layer가 forwarding에서 ComputeOps를 부르는 데까지만 정합니다.

목표 class diagram

문서 §3의 class 구조를 기준으로, 지금 이름이 있는 것은 지금 이름 그대로 썼습니다. 꼬리표 **«L5»...«L1»**은 그림 4의 층입니다. 구현 class(Cpu · Cl · Cuda · Htp ...)는 그림 2a와 같아서 상자 안 "구현:" 한 줄로 줄였습니다.

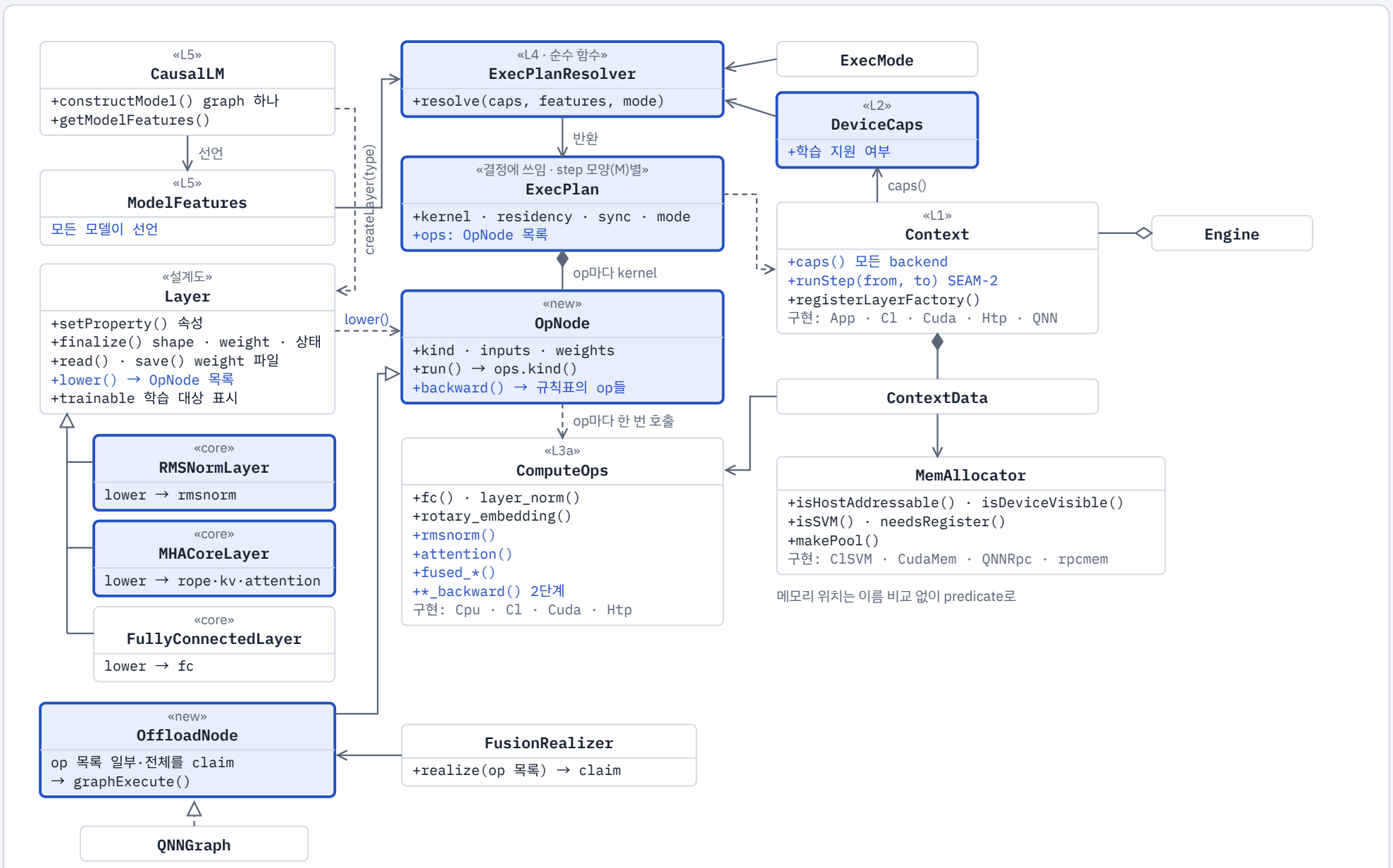


그림 5. 목표 class 구조. Layer는 설계도만 갖고, compile 때 `lower()`로 `OpNode` 목록을 만듭니다. 계산은 OpNode가 `ComputeOps`를 한 번씩 불러서 하고, backend 차이는 ComputeOps 하위 class 안에만 있습니다. 학습의 backward도 2단계에서 같은 OpNode 목록으로 돌립니다(그림 5b). prefill과 decode도 같은 목록을 `runStep` 하나로 돌리고, 실행 방식은 ExecPlan이 step 모양(M)으로 고릅니다. 모델은 graph 하나와 ModelFeatures만 내놓고, 어느 kernel · 메모리 · decode mode를 쓸지는 `ExecPlanResolver`가 정합니다. HTP의 DSP 상주 decode graph는 `HtpContext::runDecode`(offload mode) 뒤로 들어가는 것으로 봤습니다. 이 부분은 문서에 직접 적혀 있지 않은 제 해석입니다.

사라지는 class (op table로 접힘)

`LayerImplCl` `RMSNormLayerCl` `ConcatLayerCl` `ReshapeLayerCl` `TransposeLayerCl` `FullyConnectedLayerCl` `RMSNormLayerGPU` `PerLayerSliceLayerGPU` `CudaRMSNormLayer` `HtpQKVLayer` `HtpMHACoreLayer` `HtpTieWordEmbedding`

공통 graph로 합쳐지는 모델

`Gemma4HtpMoECausalLM` `Lfm2HtpMoECausalLM` `constructHtpModel()`

core로 올라오는 class

`RMSNormLayer` `ReshapedRMSNormLayer` `MHACoreLayer` `EmbeddingLayer` `SharedFullyConnectedLayer`

새로 생기거나 권한이 생기는 것

`ComputeOps::rmsnorm` `ComputeOps::attention` `fused_*` `OffloadNode` `HtpContext::caps()` `ExecPlan` 권한

목표 구조에서 layer에 남는 것

layer는 **설계도**만 맡고 **계산**은 하지 않습니다. 설계도는 compile 때 한 번 읽히고, 그걸로 만든 op 목록을 실행기가 돌립니다.

layer에 남는 것 (설계도)

- **속성**: epsilon, head 수, sliding window 같은 설정값 (`setProperty`)
- **shape**: 입력 크기로 출력 크기 계산 (`finalize`)
- **weight 선언**: 이름, 크기, 양자화 형식, 초기화, tied 공유 관계
- **weight 파일 입출력**: 파일의 어느 자리가 어느 weight인지 (`read` · `save`)
- **상태 선언**: KV cache, window ring, conv 상태가 얼마나, 언제까지 필요한지. 메모리 확보는 allocator, 값 갱신은 op가 함
- **임시 tensor와 수명**: memory planner가 메모리를 배치하는 근거
- `lower()` 새로 추가: 이 layer를 어떤 op 몇 개로 펼칠지. RMSNorm → [rmsnorm], MHA → [rope, kv_append, attention]
- **학습 설정**: 어떤 weight를 학습할지(trainable), loss인지. backward를 **어떻게** 계산할지는 layer가 아니라 op별 규칙표가 정함 (2단계)

layer에서 빠지는 것 (계산)

- **forwarding 본문** → 실행기가 op마다 `ComputeOps`를 한 번 부름
- **backend 분기** (`#if`, engine 문자열, `*_cl` · `*_gpu` · `Htp*` class) → `ComputeOps` 하위 class 안으로
- **backend 자원** (`HtpAttentionState` 같은 DSP cache handle) → 그 backend의 Context · ComputeOps로
- **실행 방식 선택** (eager · capture-replay · offload) → `Context::runStep` + Plan이 step 모양(M)으로 고름
- **모델별 decode op 목록 손코딩** (`htp_graph_gemma_build` 등) → `lower()` 결과에서 자동 생성
- **backward 계산** (`calcDerivative` · `calcGradient` 본문) → 2단계에서 op 목록으로. 그 전까지는 지금 코드 유지

지금	RMSNorm 한 층에 class가 4개(CPU · OpenCL 2 · CUDA)이고, 각자 `forwarding()` 안에서 직접 계산합니다.
목표	`RMSNormLayer` 하나가 "weight는 gamma 1개, 출력 크기 = 입력 크기, lower → [rmsnorm]"만 적습니다. 계산은 op 목록의 rmsnorm 칸이 그 backend의 `ComputeOps::rmsnorm()`을 불러서 합니다.

layer(설계도)와 Plan은 겹치지 않나

맡는 질문이 다릅니다. layer는 **무엇을 계산하나**(op, shape, weight)를 적고, Plan은 **이 하드웨어에서 어디서 · 어떻게**(kernel, 메모리 위치, 동기화, decode mode)를 정합니다. 겹치지 않으려면 경계 규칙 두 개를 지켜야 합니다.

→ **lower()는 하드웨어를 모른다.** 언제나 같은 기본 op로 펼치고, fusion · kernel · 메모리 종류는 정하지 않습니다. 그건 Plan(FusionRealizer, ExecPlan)의 일입니다.

→ **Plan은 shape · weight를 새로 정하지 않는다.** layer가 선언한 op와 tensor에 kernel과 메모리 위치만 붙입니다.

→ **실제로 겹치는 곳은 ModelFeatures 하나.** sliding_window, qk_norm, mlp 종류 같은 항목은 layer 속성에 이미 있는 정보를 모델이 한 번 더 적는 것입니다. op 목록에서 자동으로 뽑고, `decode_gpu`처럼 측정으로 정한 정책값만 따로 선언하는 게 맞습니다(문서에 없는 이 페이지의 제안).

backward도 op 목록으로, 다만 2단계

학습의 backward도 forward와 같은 실행기가 op 목록으로 돌립니다. backward 목록은 op 종류마다 정한 **규칙표**로 forward 목록에서 만듭니다. 순서는 forward를 먼저 옮기고 그다음입니다.

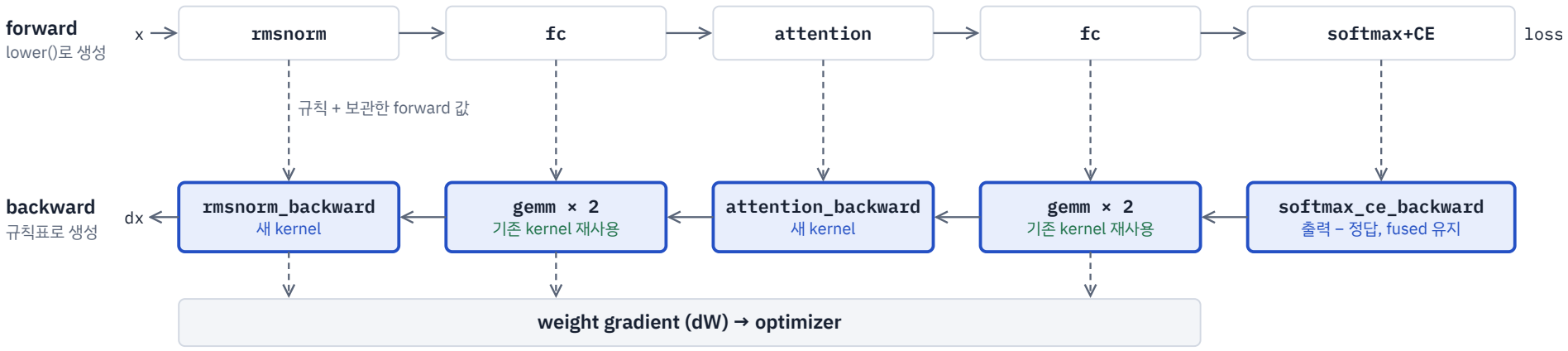


그림 5b. forward와 backward op 목록. backward는 거꾸로 돌면서 forward 때 값을 다시 쓰므로, 그 값을 언제까지 보관할지는 목록을 보고 계산합니다. fc의 backward는 지금도 gemm 두 번이라 기존 kernel을 그대로 쓰고, rmsnorm과 attention만 새 backward kernel이 필요합니다. softmax + cross-entropy는 "출력 − 정답" 규칙을 op 하나로 유지합니다.

op로 옮겨야 하는 이유

- **실행 경로가 하나로**: 안 옮기면 학습 때만 layer가 다시 계산 코드를 갖게 됨
- **fork 재발 방지**: 지금 OpenCL fork의 backward는 CPU 코드 복사, 미구현, 빈 함수가 섞여 있음
- **메모리 수명 자동 계산**: layer마다 손으로 쓰는 `TensorLifespan` 선언 대신 목록에서 계산
- **검증 재사용**: op마다 gradient check 하나로 모든 layer를 검증

forward와 동시에 하지 않는 이유

- **당장 이득이 CPU 하나**: HTP · QNN은 학습을 안 하고, CausalLM layer 17개 중 12개가 backward 미지원, OpenCL fork도 대부분 미구현
- **회귀 위험**: MNIST · ResNet · TransferLearning 같은 기존 학습 앱의 속도 · 메모리 · 수치가 바뀔 수 있어서, 75개 layer를 옮기며 gradient가 같은지 확인해야 함
- **범위가 두 배**: forward op 목록도 아직 없음

→ **방식은 규칙표.** 아무 op나 자동 미분하는 autograd가 아니라 op 종류마다 backward를 정합니다. fc → [gemm, gemm], rmsnorm → [rmsnorm_backward], softmax+CE → [softmax_ce_backward].

→ **그 사이 규칙 하나.** forward를 옮기는 동안 새로 쓰는 backward 코드는 layer가 아니라 op 단위로 씁니다.

→ **학습 못 하는 backend는 Plan이 거절.** HTP · QNN처럼 backward kernel이 없는 backend는 DeviceCaps에 그렇게 적고, 학습 모드면 ExecPlan이 거절합니다.

prefill과 decode는 한 경로로

prefill과 decode는 같은 op 목록, 같은 weight, 같은 KV cache를 쓰고, 한 번에 처리하는 줄 수(M)만 다릅니다. 그래서 op 목록 · hook · ExecPlan을 하나로 두고, 갈라지는 건 Plan이 **step 모양(M)**을 보고 고르는 결정 하나뿐입니다.

→ **경로는 하나.** hook은 `runStep(from, to)` 하나이고, "prefill 모드" · "decode 모드" 같은 이름은 두지 않습니다. 지금의 `runDecode`도 이미 prefill과 decode 양쪽에서 불립니다.

→ **kernel은 모양으로 고름.** prefill(M>1)은 계산량이, decode(M=1)는 weight를 읽는 대역폭과 호출 비용이 병목이라 맞는 kernel이 다릅니다. `accelerates_qs4cx_at_m1()`, `M > 1`일 때만 batch GEMM을 쓰는 분기처럼 이미 코드에 있는 패턴입니다.

→ **미리 묶는 방식은 M=1 반복에만.** CUDA graph replay와 HTP 상주 graph는 같은 모양이 반복될 때만 쓸 수 있습니다. M=1이 반복되면 Plan이 이 방식을 고르고, prompt처럼 길이가 바뀌면 op 단위로 돌립니다.

→ **모양별 변형은 자동 생성.** graph를 미리 묶는 backend(HTP, QNN, CUDA capture)는 같은 op 목록에서 M=1용과 M>1용 변형을 만듭니다. decode용 목록을 손으로 따로 짜지 않습니다.

문서와의 관계: `ARCHITECTURE_REFACTOR.md` § 9 결정 2는 "얇은 layer가 forwarding에서 ComputeOps를 부른다"입니다. layer가 계산을 아예 하지 않는 건(학습의 backward 포함) 그보다 한 걸음 더 나간 안이라, upstream에 가져가려면 이 결정을 바꾸는 제안으로 내야 합니다. hook을 `runStep` 하나로 두고 prefill · decode를 한 경로로 보는 것도 문서(SEAM-2 = decode hook)보다 나간 안입니다.

규칙 1a · backend 추가

새 하드웨어를 더할 때 기존 파일 수정 0

기존 모델 `.cpp`, 다른 backend 파일, `network_graph` · `layer_node`를 건드리지 않습니다.

```
새 Context + 새 MemAllocator
+ (OffloadNode 또는 ComputeOps)
```

전용 메모리 plane이 필요할 때만 enum 1개 + 문자열 1개를 추가합니다.

규칙 1b · layer 추가

기존 op에 하드웨어 지원을 더할 때 새 Layer 0개

CPU에서 돌던 **같은 Layer**가 `ComputeOps`의 whole-op virtual을 부르고, backend별 구현은 op table 안에 둡니다.

type 이름 하나에 `XLayerCl` + `CudaXLayer`처럼 class가 둘이면 위반입니다. core로 올릴지는 "차이를 parameter(ModelFeatures)로 표현할 수 있나"로 정합니다.

1 · construct	2 · compile	3 · initialize	4 · allocate	5 · prefill · decode
모델 선언	**여기서만 갈라짐**	**op table 연결**	**메모리 위치**	**입구 하나**
graph와 ModelFeatures를 내놓음. backend는 모름.	fusion realizer, resolve → ExecPlan {kernel, residency, sync, mode}.	layer마다 ContextData를 묶고 ExecPlan의 kernel을 고름.	allocator predicate로 정함. 이름 비교 없음.	prefill · decode 모두 runStep 하나로. Plan이 M을 보고 eager, capture-replay, offload 중 하나를 고름.

3 / 3

비교와 남은 일

가장 큰 차이는 **결정이 내려지는 곳**입니다. 지금도 ExecPlan을 계산하지만 그 결과에서 실제 선택으로 가는 선이 없습니다.

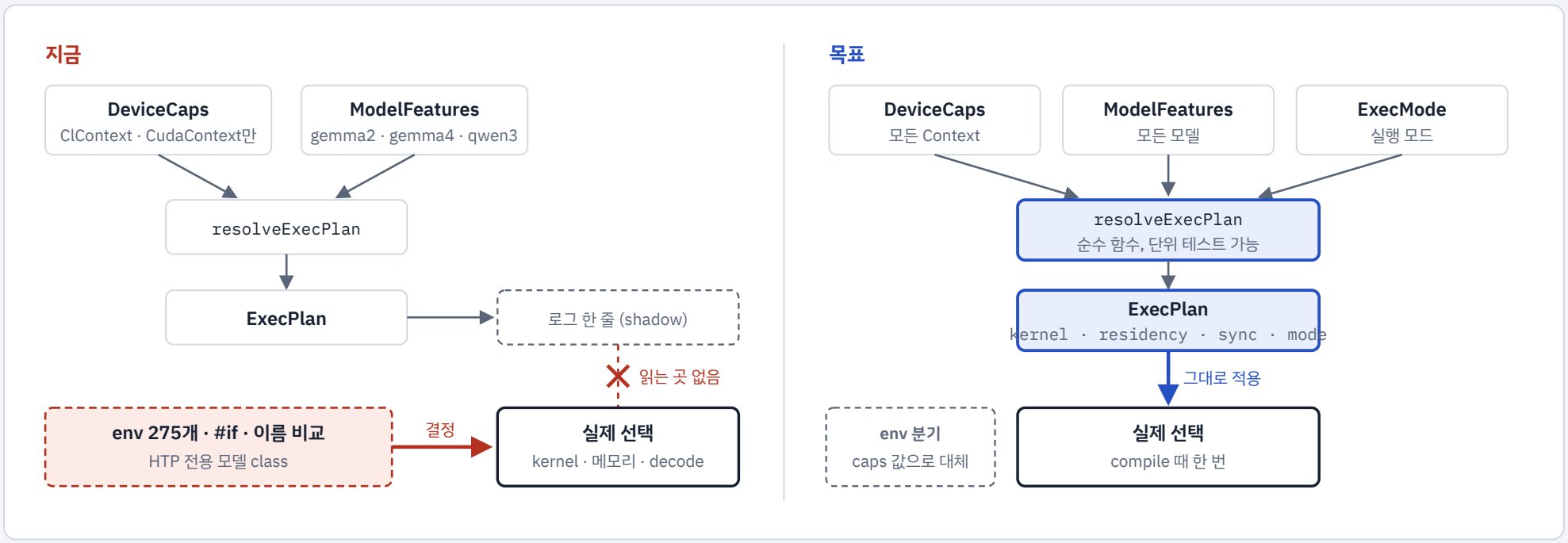


그림 6. 왼쪽의 빨간 ✗가 지금 빠진 연결입니다. 이 선을 잇는 일(ExecPlan 권한 부여)이 문서 전체에서 가장 큰 단일 변화입니다.

항목	지금 (qc_npu_rebase)	목표	상태
모델 graph	CPU · GPU 공통 graph + HTP 전용 class 2개	backend 이름 없는 graph 하나	일부
backend 결정	env 275개 (upstream 260 + qc_npu 15), `#if`, 이름 비교	compile 때 ExecPlan 한 번	로그만
layer	fork 11개: cl 5 · gpu 2 · cuda 1 · htp 3	설계도만, op당 class 하나	일부
계산 위치	layer의 `forwarding()` 안	compile 때 만든 op 목록 + 실행기	안 됨
backward	layer마다 손으로 작성 (core 75개), GPU fork는 대부분 미구현	op별 규칙표로 backward op 목록 생성 (2단계)	안 됨
op table	`rotary_embedding` · `layer_norm` · `fc` · `swiglu_fp32` · `sdpa_q_kvcache`(int8 KV)	+ `rmsnorm` · `attention` · `fused_*`	일부
mha_core	6,117줄, 안에 OpenCL · CUDA arm	arm은 op table로 이동	안 됨

항목	지금 (qc_npu_rebase)	목표	상태
DeviceCaps	ClContext · CudaContext만	모든 Context	일부
메모리 위치	predicate 있음 + 이름 비교 2곳 (`'qnn'`, `'gpu-svm'`)	predicate만	거의
실행 방식	`runDecode`(Cuda · Htp)가 prefill · decode를 모두 받음, HTP 단일 token decode는 hook 밖(`npuDecodeTokenId`), QNNGraph는 QNN 전용	hook 하나(`runStep`), Plan이 step 모양(M)으로 방식 선택, OffloadNode	일부
layer 작성 API	공개 API 없음, AppContext downcast 4곳	`registerLayerFactory`만	일부
fusion	FusionRealizer 있음, 범용 fused virtual 없음	realizer + `supports_fused_*()`	일부

▼ **근거 (file: symbol)**

- ExecPlan 로그만: `nntrainer/context.h: ExecPlan` 주석 "no decision site reads it", `causal_lm.cpp`의 `(shadow)` 로그
- env 수: cl_operations 105, cuda 44, Applications/CausalLM 84, htp_backend 1 (영역별 고유 이름 수, 영역끼리 겹침 있음)
- layer fork 등록: `cl_context.cpp` (`*LayerCl`), `transformer.cpp` (`RMSNormLayerGPU`, `PerLayerSliceLayerGPU`, `CudaRMSNormLayer`), `gemma4_htp_causallm.cpp` · `lfm2_htp_moe_causallm.cpp` (`HtpQKVLayer`, `HtpMHACoreLayer`, `HtpTieWordEmbedding`)
- op table: `nntrainer/tensor/cpu_backend/compute_ops.h`, fused는 `gemm_qs4cx_fused_swiglu_fp32` 하나
- caps(): `cl_context.h`, `cuda_context.h`에만 override
- 이름 비교: `memory_pool.cpp` (`'qnn'`), `kv_cache_manager.cpp` (`'gpu-svm'`)
- downcast: `gemma4_moe_causallm.cpp`, `lfm2_causallm.cpp` (upstream), `gemma4_htp_causallm.cpp`, `lfm2_htp_moe_causallm.cpp` (qc_npu)
- HTP 모델 분기: `gemma4_causallm.cpp`의 `gemma4_execution_profile == "htp_fused"`
- decode hook: `Context::runDecode` (`context.h`), override는 `cuda_context.h` · `htp_context.h`, `nntrainer/npu/npu_executor.h`

backend별 구조 불일치

위 표를 backend 단위로 풀었습니다. 각 줄 오른쪽 §는 어긴 문서 조항이고, 회색 줄은 근거 위치입니다. 초록 줄은 그 backend에서 이미 목표 모양인 부분입니다.

공통 · CPU `core, engine=cpu`

9건

맞는 것: `CpuComputeOps`가 모든 backend의 기준 table. FusionRealizer가 compile chain에 있음. `engine=` 이름은 등록된 Context 기준으로 검증.

rmsnorm · attention virtual이 op table에 없음. 그래서 RMSNorm 구현이 5벌이고, 그중 op table을 부르는 것은 하나도 없습니다. attention은 CPU 경로까지 `mha_core.cpp` 안에 있습니다.

§4 · §10

`rms_norm.cpp` 208줄 · `reshaped_rms_norm.cpp` 345 · `rms_norm_gpu.cpp` 266 · `rmsnorm_layer_cl.cpp` 302 · `cuda_rmsnorm_layer.cpp` 182

type 이름 하나에 class 여러 개. 같은 type을 engine마다 다른 class로 등록해서 갈아 끼웁니다. RMSNorm 4 class, `fully_connected` 2 class.

§1b

`transformer.cpp`의 `tryRegister("gpu" | "cuda", …)`, `cl_context.cpp`, `cuda_context.cpp`

LLM 기본 layer가 아직 앱에 있음. 문서가 PROMOTE로 판정한 rms_norm, reshaped_rms_norm, mha_core, embedding_layer, shared_fully_connected_layer가 그대로 `Applications/CausalLM/layers/`에 있습니다.

§4

core `nntrainer/layers/llm/`에 올라간 것: swiglu, geglu, lm_head, qkv, logit_softcapping, scalar_multiply, tie_word_embedding

backend 선택이 process 전역 문자열 하나. API 요청이나 `NNTR_ENGINE`을 `causallm_engine()`이 읽고, layer도 이 값을 직접 읽어 분기합니다. 문서는 backend별 전략을 Context hook 뒤에 두라고 합니다.

§6

`mha_core.cpp`: `nntr_engine_env() == "cuda"` · `transformer.cpp`: `causallm_engine() == "cuda"`일 때만 layer 등록

모델 graph에 backend 이름. gemma2 · gemma4가 mha_core에 `gpu_decode_attn`, `gpu_decode_rope`, `gpu_ohwi_rope` 속성을 넣습니다. 값은 ModelFeatures에서 오지만 키 이름이 backend를 가리킵니다.

§7

`gemma2_causallm.cpp`, `gemma4_causallm.cpp`의 `withKey("gpu_decode_attn", …)`

ModelFeatures 선언이 일부 모델만. gemma2 · gemma4 · qwen3(+파생)만 `getModelFeatures()`를 채웁니다. gemma3, qwen2, lfm2, lfm2_moe, gpt_oss는 기본값이라 resolver가 이 모델들을 구분할 수 없습니다.

§7

`getModelFeatures` override: `gemma2_causallm.h`, `gemma4_causallm.h`, `qwen3_causallm.h`

ExecPlan은 로그만. CPU의 `AppContext`도 `resolveExecPlan(caps())`를 찍기만 합니다. CausalLM 쪽 env는 84개(models 25, layers 40)입니다.

§10

`app_context.cpp`, `causal_lm.cpp`의 `(shadow)` 로그

fusion이 절반만 있음. FusionRealizer는 activation을 layer 내부 `fused_activation` 속성으로 넘깁니다. 문서가 말하는 `fused_*` virtual + `supports_fused_*()` 짝은 없고, `bn_realizer`는 chain 밖에 있습니다.

§8

`nntrainer/compiler/fusion_realizer.h`, `neuralnet.cpp: NeuralNetwork::compile`

layer 작성 API가 없음. 앱 layer는 내부 base를 상속하고, CPU 모델 2곳이 등록하려고 `AppContext`로 downcast합니다.

§10

`gemma4_moe_causallm.cpp`, `lfm2_causallm.cpp`의 `static_cast<nntrainer::AppContext *>`

OpenCL `engine=gpu · Intel Xe, Adreno` 7건

맞는 것: caps()를 가진 두 backend 중 하나. FC는 `ComputeOps::fc`를 거침. `ClSVMAllocator`가 predicate 구현.

layer fork 7개. `LayerImplCl` 아래 4개(RMSNorm, Concat, Reshape, Transpose), `FullyConnectedLayerCl`, 앱의 `RMSNormLayerGPU` · `PerLayerSliceLayerGPU`.

`nntrainer/layers/cl_layers/`, `fc_layer_cl.h`, `rms_norm_gpu.h`, `per_layer_slice_gpu.h` §1b · §4

mha_core 안에 OpenCL 분기 27곳. layer가 `cl_svm_map_force` · `cl_queue_finish`를 직접 부릅니다. op table에 attention virtual 이 없어서 생긴 결과입니다.

`mha_core.cpp`의 `#if defined(ENABLE_OPENCL)` 27곳 §4 · #4451

env 105개가 kernel 경로를 고름. 각 변수가 문서 말로 "아직 안 쓴 resolver cell"입니다.

`tensor/cl_operations/`의 고유 이름 수. prefix별: NNTR_FLASH 24, NNTR_FC 22, NNTR_V8C 21 §10

kernel이 caps()를 직접 읽음. 능력에서 결정으로 가는 방향은 맞지만, ExecPlan을 거치지 않아서 resolver를 단위 테스트로 검증할 수 없습니다.

`ClContext::Global().caps().dpas` · `.image_v8c` · `.subgroups`: `blas_kernels.cpp`, `attention_kernels.cpp`, `blas_kernel_interface.cpp` 10곳 §2

KV backing과 coherence drain은 caps로 못 정함. image2d vs buffer는 vendor id로, SVM coherence drain은 보수적 env로 정합니다. coherence drain은 문서가 첫 resolver cell로 지목한 항목입니다.

`nntrainer/context.h: ExecPlan` 주석 §6

KV cache가 allocator 이름을 비교. 메모리 위치는 predicate로 정해야 합니다.

`kv_cache_manager.cpp`: `getName() == "gpu-svm"` §3

backward가 fork마다 따로이고 대부분 비어 있음. FC는 CPU와 같은 코드를 복사했고, RMSNorm · transpose는 지원하지 않으며, concat은 빈 함수라 학습하면 오류 없이 gradient가 틀립니다.

`FullyConnectedLayerCl::calcDerivative` = `dot_deriv_wrt_1` (CPU와 동일), `ConcatLayerCl::calcDerivative` 본문 없음, `TransposeLayerCl`는 throw §1b

CUDA `engine=cuda` 4건

맞는 것: `CudaComputeOps : CpuComputeOps`로 fallback. `CudaContext::runDecode`가 decode hook 자리에 있음. `CudaMemAllocator`가 predicate 구현, `isSVM() == false` 계약 지킴.

RMSNorm fork를 앱이 등록. core의 `CudaRMSNormLayer`를 앱 `transformer.cpp`가 engine 문자열을 보고 등록합니다.

`nntrainer/layers/cuda_layers/cuda_rmsnorm_layer.h`, `transformer.cpp` `tryRegister("cuda", …)` §1b

FC는 OpenCL class를 빌려 씀. CudaContext가 `fully_connected`로 `FullyConnectedLayerCl`을 등록합니다. 이름부터 backend가 어긋나고, core FC와 별개 class입니다.

`cuda_context.cpp`: `createLayer<FullyConnectedLayerCl>` §1b · §4

mha_core 안에 CUDA 분기 11곳. layer가 `cuda_attention.h` · `cuda_rope.h`를 include하고, `cudaMalloc` · `cudaMemcpy`와 stream capture 상태를 직접 다룹니다.

`mha_core.cpp`의 `#if defined(ENABLE_CUDA)` 11곳, `StreamManager::Global().isCapturing()` §4 · §6

decode capture 여부를 env가 정함. eager냐 capture-replay냐는 문서상 DeviceCaps와 실행 모드로 resolver가 정할 일인데, 지금은 `NNTR_CUDA_GRAPH` · `NNTR_CUDA_M2B`가 정합니다.

`nntrainer/cuda/` 안의 env 44개, `cuda::decodeGraphEnabled()` §6 · §9.5

HTP `engine=htp · qc_npu` 9건

맞는 것: `HtpComputeOps : CpuComputeOps`로 fallback, 문서가 "검증된 경로"로 인용. 설정은 `htp` · `moe` schema라 env 읽기는 1개. `NpuExecutor`는 vendor 이름 없는 core interface. `HtpContext::runDecode`가 decode hook 자리에 있음.

모델 graph가 따로 있음. HTP 전용 subclass 2개에 더해, Gemma4는 공통 class 안에서도 `HTP_GRAPH` 플래그로 graph 생성이 갈라집니다. §1a · §7

`Gemma4HtpMoECausalLM`, `Lfm2HtpMoECausalLM`, `Gemma4Transformer::constructModel` → `constructHtpModel()`, `gemma4_execution_profile == "htp_fused"`

같은 모델을 두 번 기술. prefill용 layer graph와 따로, decode용 op 목록을 모델마다 손으로 짭니다. 모델별 builder가 backend header 안에 있습니다. §1a · §6

`htp_graph_desc.h`: `htp_graph_gemma_build()`, `htp_graph_lfm2_build()`, 호출은 `Gemma4HtpMoECausalLM::load_weight` · `lfm2_htp_moe_causallm.cpp`의 `set_decode_graph_desc(words)`

---

layer type 3개 추가. `htp_mha_core`는 type 이름만 바꾼 19줄 subclass입니다. §1b

`htp_qkv_layer`, `htp_mha_core`, `htp_tie_word_embeddings`

---

core attention class가 HTP 상태를 직접 들고 있음. 중립이어야 할 `nntrainer/layers/llm/`의 class가 HTP 이름의 member와 method를 갖습니다. §1b · §9.2

`fp16_attention.h`: `std::unique_ptr<HtpAttentionState>`, `beginHtpPrompt()`, `setHtpPrefillSeed()`, `htpDecodeAttention()`

---

op table이 HTP 자원 관리 창구가 됨. ComputeOps virtual이 104개에서 161개로 늘었고, 상당수는 연산이 아니라 weight 등록 · 해제 · prefetch입니다. 중립 interface에 vendor 이름도 들어갔습니다. §9.2

`register_qs4cx_weight`, `set_fc_wh_file`, `release_prefill_scratch`, `prefetch_qs4cx_wh_experts_end`, `htpExpertPool()`

Context에 두 번째 NPU 입구. base Context에 `npuExecutor()`가 추가됐고(upstream에는 없음), 모델 코드가 직접 부릅니다. 문서는 decode 전략을 hook 하나(runDecode) 뒤에 둡니다. §6 · §9.5

`context.h: Context::npuExecutor`, 호출: `causal_lm.cpp`, `lfm2_moe_layer.cpp`, `quick_dot_ai_api.cpp`

---

op table과 executor가 한 class. 문서 층으로는 L3a(op table)와 L3b(decode · offload)가 한 객체에 있습니다. 문서가 직접 금지하지는 않아서 제 판단입니다. §2 (해석)

`htp_compute_ops_impl.h`: `class HtpComputeOps : public NpuExecutor, public CpuComputeOps`

---

caps() 없음, 메모리는 allocator 밖. resolver가 HTP를 고를 근거가 없고, rpcmem arena는 backend 안에서 따로 관리됩니다. model tensor는 기본 CPU allocator를 씁니다. §5 · 2, 4단계

`htp_context.h`에 `caps()` override 없음, `htp_rpcmem.h`

---

downcast +2, env +15. env는 대부분 진단용입니다(`NNTR_ACT_DUMP`, `NNTR_OP_TIME` 등). §10

`gemma4_htp_causallm.cpp`, `lfm2_htp_moe_causallm.cpp`

QNN `engine=qnn` 3건

맞는 것: `QNNGraph`가 문서의 whole-graph offload 원형. vendor SDK를 dlopen으로 붙이는 등록 방식.

---

op table도 caps()도 없음. QNNContext는 ComputeOps를 묶지 않고, graph 밖 연산은 전역 CPU table로 떨어집니다. §5 · 2단계

`qnn_context.h`

QNNGraph가 QNN 전용. 범용 `OffloadNode`로 일반화되지 않아서 다른 NPU가 재사용할 수 없습니다. §5 · 3단계

`nntrainer/qnn/jni/QNNGraph.cpp`

메모리를 이름으로 구분. `QNNRpcManager`는 `needsRegister()` 등 predicate를 override하지 않고, pool은 이름으로 분기합니다. §3 · §5 4단계

`memory_pool.cpp`: `getName() == "qnn"`

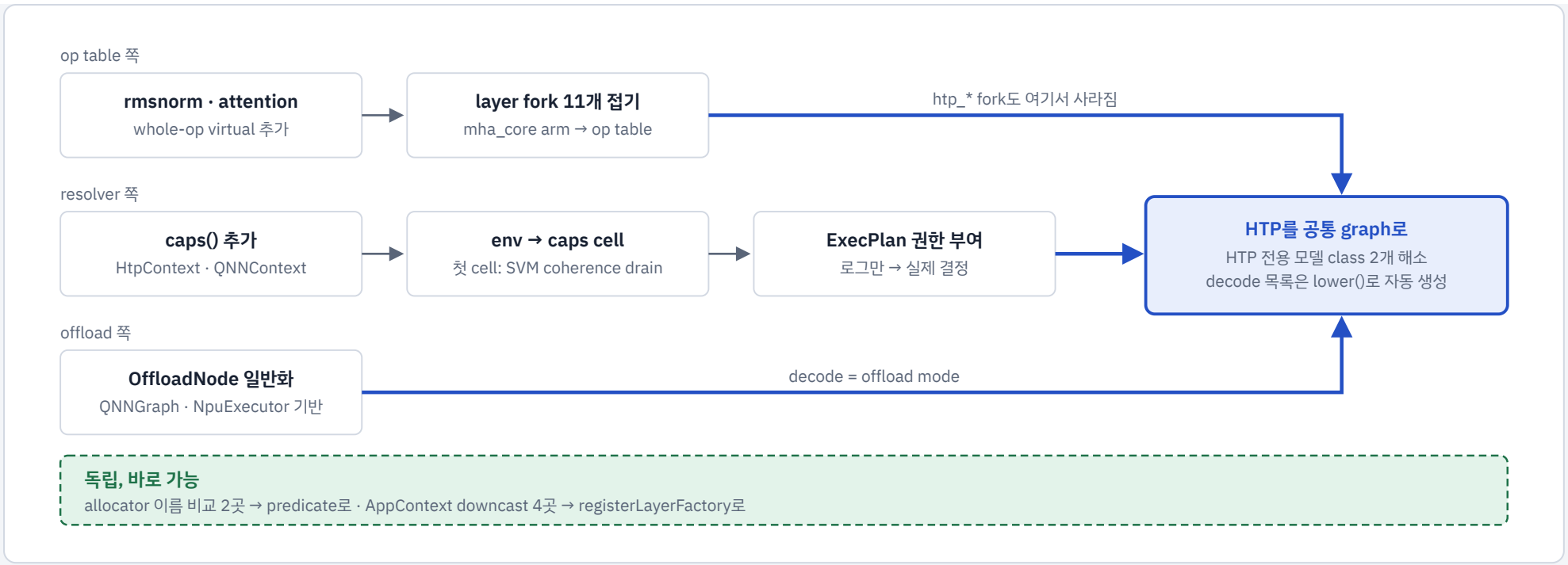


그림 7. 세 줄이 모두 끝나야 HTP를 공통 graph로 합칠 수 있습니다. 왼쪽 위 "virtual 추가"가 가장 많은 것을 풀어줍니다: fork 접기, mha_core 정리, `rms_norm` · `mha_core`의 core 승격이 모두 여기에 걸려 있습니다.

제안 순서

문서는 일부러 순서나 일정을 담지 않습니다. 아래 순서는 그림 7의 의존 관계에서 뽑은 제안입니다.

1. **작은 것 먼저.** 이름 비교 2곳과 downcast 4곳. 다른 일과 얽히지 않고, 문서가 "새로 더하지 말 것"으로 못 박은 항목입니다.
2. **rmsnorm · attention virtual.** 이게 들어가야 fork 11개를 접고 `mha_core.cpp`의 OpenCL · CUDA arm을 op table로 옮길 수 있습니다. 그 전까지는 "fork 더 만들지 말고, 본체는 helper 하나로" 규칙만 지키면 됩니다. layer에 `lower()`를 붙이고 op 목록 실행기를 만드는 일도 이 단계에 들어갑니다.
3. **ExecPlan에 권한 주기.** HtpContext · QNNContext에 caps()를 달고, env 변수를 하나씩 caps에서 나온 값으로 바꿉니다. 문서가 지목한 첫 cell은 SVM coherence drain입니다.
4. **마지막에 HTP 공통 graph.** 위 셋과 OffloadNode가 있어야 Gemma4 · LFM2의 HTP 전용 class를 공통 graph + ExecPlan mode로 바꿀 수 있습니다. decode용 op 목록을 `lower()` 결과에서 자동으로 만들면 `htp_graph_gemma_build` 같은 손코딩이 사라집니다. 단일 token decode가 모델 코드(`npuDecodeTokenId`)에서 직접 들어가는 경로도 `runStep` 뒤로 옮깁니다. 바꾼 뒤에는 token-identical 확인이 필요합니다.
5. **학습 쪽은 그다음 backward.** forward op 목록이 안정되면 op별 backward 규칙표를 만들고, core layer의 `calcDerivative`를 하나씩 옮깁니다. 옮길 때마다 gradient check로 기존 결과와 같은지 확인합니다.

숫자는 2026-11-06에 `d061fcd20` tree를 grep해서 셌습니다. env 수는 `getenv("이름")`의 고유 이름 수이고, `nntrainer/` + `Applications/CausalLM/`에서 test · xgrammar를 뺐습니다.

class diagram은 backend 구조에 걸리는 class만 골랐고, 상속 관계는 header의 class 선언에서 뽑았습니다. 목표 class diagram은 문서 §3을 따르되 HTP decode 위치는 제 해석입니다.

문서 §10은 downcast를 1곳으로 적었지만 upstream main에 이미 2곳, 이 브랜치에 4곳 있습니다. 빌드나 실행으로 확인한 내용은 없습니다.